

# Report — BasicRLAgent

Agent to play Gomoku

Name: Willian Franco da Nóbrega

GitHub: [willianfnobrega/AI\\_STUDIO](https://github.com/willianfnobrega/AI_STUDIO) ([https://github.com/willianfnobrega/AI\\_STUDIO](https://github.com/willianfnobrega/AI_STUDIO))

## 1. How the agent works:

### 1.1 Initialize and parameters

He is created with 3 symbols (itself, empty, enemy), and uses the following ones for calculations:

- Q-table (`q_table`): For each state of the game already visited, stores its learned outcome for each possible movement.
- Learning rate ( $\alpha = 0.2$ ): How the agents update its old value at each movement progress.
- Discount Factor ( $\gamma = 0.9$ ): I declared, but didn't really find how to use in this application...
- Epsilon ( $\epsilon = 0.3$ ): Probability of an agent to choose a random move instead of the know one
- `games_played`: game played counter, used for epsilon decay;
- `last_state` / `last_action`: stores last state and movement for the continuity of the match.

### 1.2 State Representation

Each turn, the table (numpy matrix) is converted into a state by the function `_state_key`:

```
return tuple(int(cell) for cell in board.flatten())
```

The board is converted into a tuple of integers that works as keys in the `q_table` dictionary, this works for the agent to identify and retrieve learned values for each setup the board had previously.

### 1.3 Valid Moves

The valid moves does not return all empty places in the board, instead it loads the neighbour ones (8 directions) of some piece already place. This limits the active regions on the game to avoid overwhelming the agent. If the board is fully empty, all places are valid.

### 1.4 Victory Detec

The method `_winning_move()` will verify for a given movement and symbol if there is a sequence of 5 pieces already in any direction. The algorithm goes until 4 places in each direction from the position already tested, accumulating the counting and stopping when it finds a empty position or different piece.

### 1.5 Decision Making: `play()`

Each turn follows this order of decision:

- First play: if the board is empty, goes for the center.

- Immediate Win: for each valid movement, simulates the play of the agent and checks if its win. If so, executes the movement.
- Blocking: for each valid movement, simulates the play of the enemy and checks if its his win, if so, blocks the movement. This one must be upgraded to also detects when a sequence of open 3 is made.
- Policy and greedy: if none of the cases applies, calls `_choose_move()` to select the movement via q-table or random.

Before returning any movement, the agent stores the actual position and the actual state, and the choosen action, for later usage in its update.

### 1.6 `_choose_move()`

With certain probability, the agent returns a random move between the valid ones (exploration). With probability (1 - certain), checks the Q-table and finds the value of higher value. If it is a tie, gets the movement that's closer to the board. If the q-table has no information on the current state, the standard value is 0 for all moves.

### 1.7 `on_game_over()`

At the end of each match, the agent gets the reward and updates the Q-table:

- Win – reward = +1.0
- Lose - reward= -1.0
- Tie - reward= 0.0

Then, the counting matches is incremented and epsilon is reduced:

This makes the agent explore less and look for the higher value more often, getting experience. The minimum epsilon is 0.05, granting a small chance of exploration. At last, `last_state` and `last_action` are cleaned for the next round.

## 2. Strong Points:

- Win and immediate block are granted: the agent never loses for not seeing an obvious win or a necessary block.
- Centralized opening: playing in the middle is a rule based option that offers advantage in the gomoku game.
- Reduced action space: restricting the moving area makes the agent focus on the surroundings and do not take too long to simulate.
- Exploration decay: epsilon decay makes the agent evolve from exploratory to exploitation naturally.
- Prefers central positions: if it is a tie, the agent prefers positions closer to the center, also an advantage in Gomoku.

## 3. Weak Points

- Needs more learning to be effective.
- I could not find a way to use the (Y) factor the we declared...
- No immediate reward: the agent could decide better if getting feedback during the match.
- The q-table should be saved and used for the future.

## 4. Conclusion

The agent is functional and combine rule-based with reinforcement learning, granting a reasonable behaviour at least in the beginning of each match. It works as a learning agent, but to play better the points highlighted before need to be treated. When playing against the minimax agent I created, the agent lost everytime. A more dynamic learning and bigger pre-training would help as well.